

In this part, you need to train a linear binary classification model $F(x) = \langle w, x \rangle$ with logistic regression (for simplicity, the bias term b is omitted here). For training samples $\{(x_i, y_i)\}_{i=1}^n$ where $x_i \in \mathbb{R}^d$ and $y_i \in \{0, 1\}$, you need to find a vector $w \in \mathbb{R}^d$ that minimizes the following loss function:

$$\ell(w) = \frac{1}{n} \sum_{i=1}^n \left[\log(1 + e^{\langle w, x_i \rangle}) - y_i \cdot \langle w, x_i \rangle \right]$$

which is the negative log likelihood. Minimizing this loss function is a convex optimization problem, which means that in most cases, there exists a unique minimizer. Here, you need to find this minimizer with gradient descent, which runs as follows:

Algorithm 1 Gradient Descent

Require: Learning rate $\eta > 0$, stopping criterion $\rho > 0$

Initialize w

while True **do**

 Compute gradient $\nabla_w \ell(w)$

 If $\|\nabla_w \ell(w)\| < \rho$ then break

$w \leftarrow w - \eta \cdot \nabla_w \ell(w)$

end while

Task: In `reg.py`, implement three functions: `compute_loss` for computing $\ell(w)$, `compute_grad` for computing $\nabla_w \ell(w)$, and `train` for training the model with gradient descent.